

LFP Cell Physical Modeling & Parametrization in PyBaMM

A123 ANR26650M1B — DFN Model Development, Parameter Optimization, and Validation

DFN model simulation.PY

Step 1 & 2 - Building the DFN Model and Running It on the HPPC Data

What this code does

This file does three things in order:

1. Loads the HPPC test data and cleans it up.
2. Builds a DFN (Doyle-Fuller-Newman) model in PyBaMM using the Prada2013 parameter set, then scales it to match our 2.5 Ah cell.
3. Runs the model using the real HPPC current profile, and compares the simulated voltage to the measured voltage.

This is the "before optimization" version of the model -the most plain setup possible. It uses Prada2013's built-in OCP curves directly (no literature OCP swap yet), no fitted stoichiometry window, no capacity-matched electrodes, and a manually guessed initial SOC. Basically, nothing cell-specific has been tuned yet except the 2.5 Ah capacity scaling.

Methodology and assumptions

Loading the data: The HPPC sheet is read from the Excel file. A few basic cleaning steps are applied - timestamps are sorted, any duplicate or repeated timestamps are removed, and time is shifted so the test starts at $t = 0$. The current column is also flipped in sign, because PyBaMM uses the opposite convention from the test equipment (PyBaMM treats a positive current as discharge, but the raw data logs positive current as charge).

Handling the rest periods: The HPPC test has long rest periods between pulses. During a rest, the current doesn't change, so there's no need to feed the solver every single data point - that would waste time without adding useful information. A function called `decimate_rests` keeps only one sample every 5 seconds during long flat rests, while keeping every point during the active pulses. This keeps the solver input smaller and faster without losing any real behavior.

Model choice: The DFN model is used, as required by the assignment. Three options are set:

- `thermal`: isothermal - because the data doesn't cover different temperatures, so we're told to ignore thermal effects.

- surface form: false and particle: Fickian diffusion - standard DFN settings for simple Fickian particle diffusion, a reasonable default before any tuning is done.

Parameter set choice: Prada2013 was chosen because it's PyBaMM's built-in parameter set for a graphite/LFP (LiFePO₄) chemistry, which matches our A123 cell's chemistry. It's the closest match available in PyBaMM's library. At this stage, its OCP (open-circuit potential) functions and its stoichiometry window are used exactly as PyBaMM ships them - they are not yet fitted to our actual cell.

Capacity scaling: Prada2013 is built for a 2.3 Ah cell, but our cell is rated 2.5 Ah. Instead of changing every parameter by hand, only the electrode width is scaled by 2.5/2.3. Electrode width controls how much active material is in the cell, so scaling it up proportionally increases the cell's capacity to match 2.5 Ah without disturbing the underlying material properties (diffusivity, kinetics, etc.), which stay as Prada2013 defined them.

Voltage cut-offs: Set to 2.0 V and 3.8 V, matching the datasheet's HPPC pulse voltage limit (3.8 V) and minimum voltage (2 V), so the simulation can follow the real test without stopping early. [A123 ANR26650M1B datasheet - Lithium Werks](#)

Initial SOC: Set to a fixed 96.49%, using PyBaMM's built-in set_initial_stoichiometries function with known_value="cell capacity". This is a manual guess, not something backtracked from the real measured voltage - and because it's mapped onto Prada2013's own default stoichiometry window (not one fitted to our cell), the mapping between "96.49% SOC" and "starting voltage" doesn't necessarily match where our real cell actually started.

Solver: CasadiSolver in "safe" mode, with tight tolerances (1e-6). This solver is chosen because it's stable for stiff battery models and less likely to fail partway through a long 25-hour simulation like the HPPC test - though it is more numerically conservative (and slower) than the IDAKLU solver used in later steps.

Results

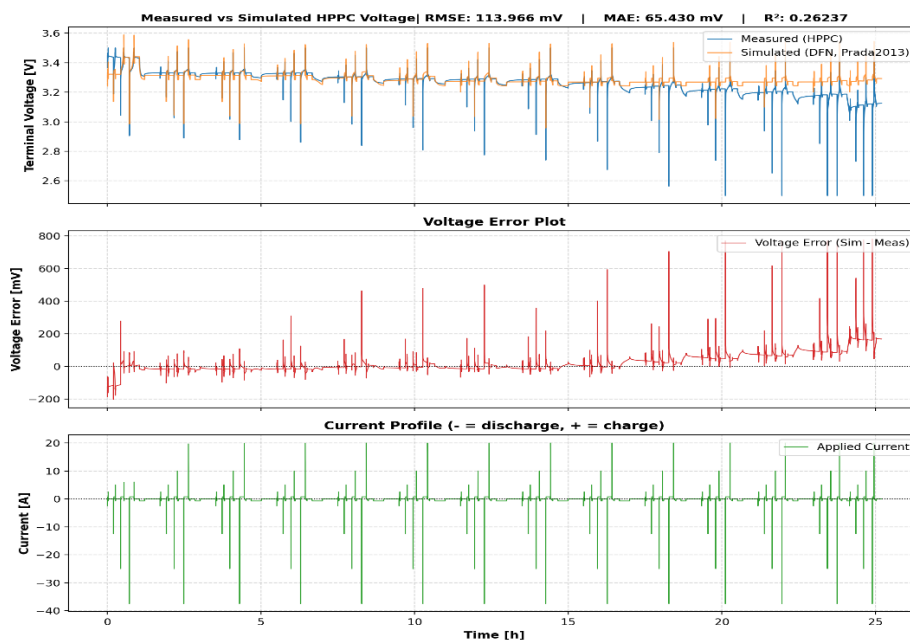
Metric	Value
RMSE	113.97 mV
MAE	65.43 mV
Max Error	781.83 mV
R ²	0.262
Points scored	55,460 (full profile)
Solver termination	final time (completed the whole test)

An R^2 of 0.262 means the model only explains about 26% of the actual voltage variation - it's tracking something close to the average voltage with large systematic errors, rather than genuinely following the cell's behaviour.

Why RMSE was chosen as the main metric: RMSE (Root Mean Square Error) is a good general accuracy measure because it squares the errors before averaging, which makes big mistakes count more than small ones. In a battery model, a big miss (like the model being way off during a heavy discharge pulse) matters a lot more than a lot of tiny 1 mV errors, so RMSE reflects that. MAE and Max Error are also reported for a fuller picture - MAE shows the "typical" error size, and Max Error shows the worst single moment.

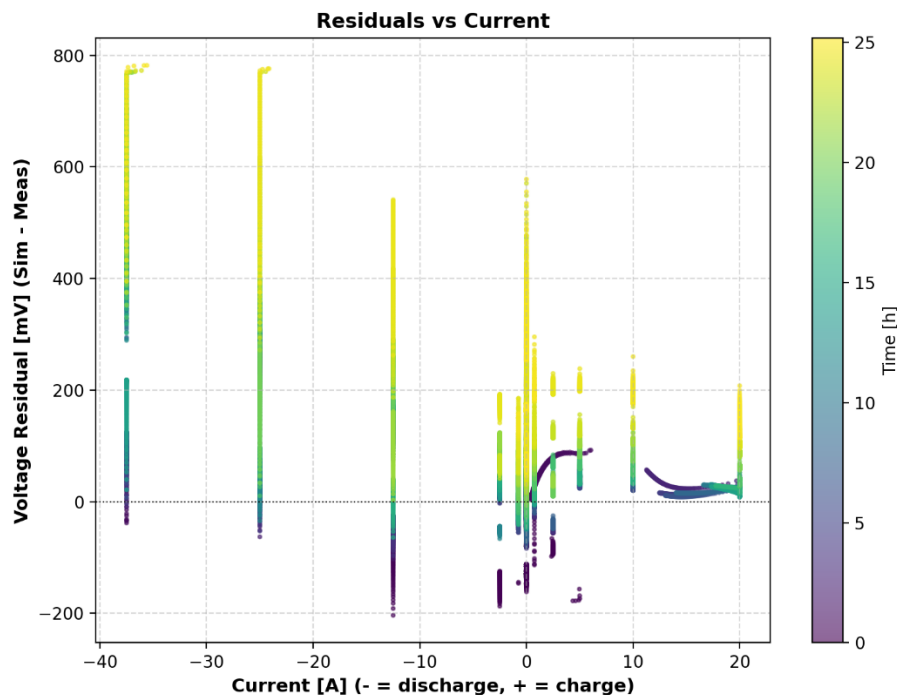
Explaining the plots

Plot 1: Measured vs Simulated Voltage + Error



- The measured voltage slowly drops from about **3.30 V at the start** to **3.10–3.15 V by 25 h**.
- The simulated voltage stays almost constant around **3.28–3.30 V**, so the model does not follow the actual drop in SOC.
- This happens because the model starts with a fixed **96.49% SOC**, but the SOC–stoichiometry range from Prada2013 was not fitted to our cell.
- The Prada2013 OCP curve is also flatter in the LFP region, so the model gives a higher voltage than the real cell at the same SOC.
- During the first few hours, the simulated discharge pulses are deeper than the measured ones, but later the simulated voltage becomes higher mainly because of the growing **SOC error**.

Plot 2: Residuals vs Current



- The residuals form clear groups at the different current levels because the HPPC test uses fixed current values.
- For discharge currents such as **-40 A**, **-25 A**, and **-12 A**, the early points are close to zero, while the later points become strongly positive.
- Even at **0 A**, the error increases with time, reaching around **100–200 mV**, showing that the problem is not caused only by resistance or current effects.
- During charging, the model already gives higher voltage than the measured value from the beginning, and this error becomes larger as the charging current increases.
- Overall, the plots show that **SOC tracking is the main problem**, while the charging region also has a separate issue related to **kinetics and/or contact resistance**.

Key observations

1. The model does reasonably well early in the test but gets progressively worse as SOC drops - this is a sign that the model's electrical/kinetic behaviour isn't the main issue, since it would be consistently wrong from the start if it were.
2. The errors are largest during high-current pulses, meaning the model isn't fully capturing the voltage drop during fast discharge - likely from the default (untuned) diffusion and kinetics parameters.
3. The growing gap over time strongly suggests the model's state-of-charge tracking is drifting away from the real cell's state-of-charge - most likely because the default Prada2013 parameter set doesn't perfectly match our actual OCV curve, causing errors to build up over a full 25-hour test.

4. Discussion

This baseline confirms the model runs successfully end-to-end (it completed all 25 hours without crashing) and gives a reasonable starting point, but 114 mV RMSE and an R^2 of only 0.26 tell us the plain Prada2013 defaults aren't accurate enough on their own. This result is the reason later steps focus on: (1) fixing the OCV/stoichiometry curve so the model's SOC tracking matches the real cell more closely, and (2) tuning the kinetic and diffusion parameters, since the residual pattern (worse at high current, worse over time) points to both of these being off in the default parameter set.

Step 3, Part A — step3_safari_ocp.py: Fixing the OCV Curve and Capacity (HPPC)

What this code does

This is the main improvement step over the Step 1&2 baseline. It keeps the same DFN model but replaces two things that were causing the baseline's drift: the OCV curve (using literature OCP functions instead of Prada2013's built-in ones) and the electrode capacities (rescaled to actually match the cell's real 2.5 Ah). It then re-runs the same HPPC test to see how much this fixes.

Methodology and assumptions

Why change the OCP curves: The Step 1&2 baseline used Prada2013's built-in OCP (open circuit potential) curves for the anode and cathode. But Prada2013 was built for a different physical cell than ours, so its OCP curves don't necessarily match how our actual A123 cell's voltage behaves at each state of charge. Since the case study explicitly allows using OCP curves from the literature, [the Safari & Delacourt \(2011\)](#) OCP functions are used instead — these are published equations specifically for a graphite/LFP cell chemistry, which matches ours. This is a literature-external resource, and it's justified because the assignment permits it and because our cell is also a graphite/LFP chemistry.

Fitting the stoichiometry window: OCP functions alone aren't enough - you also need to know which stoichiometry range (x_0 to x_{100} for the anode, y_0 to y_{100} for the cathode) corresponds to 0–100% SOC for this specific cell. This is found by fitting the model's predicted OCV (built from the two OCP curves) against the cell's actual measured OCV table (OCV_25C_Clean_Monotonic.xlsx) using least-squares curve fitting. The result was:

- $x_0 = 0.0015$, $x_{100} = 0.7760$ (anode range)
- $y_0 = 0.9471$, $y_{100} = 0.0231$ (cathode range)
- Fit accuracy: 12.58 mV RMSE across the full SOC range

This fit is checked against the paper's own reported stoichiometry values ($x_{\max} \sim 0.80$ – 0.90 , $y_{\min} \sim 0.03$, $y_{\max} \sim 0.76$ – 0.86) and falls within that expected range, so the fit is physically reasonable, not just a mathematical accident.

Monotonicity pre-flight check: Before running anything, the code checks that the fitted OCV curve only goes in one direction (increasing) across the whole SOC range. If it dipped up and down, that would mean two different SOC values give the same voltage, which would make it impossible to correctly figure out SOC from voltage. This check passed with 0 violations.

Capacity matching: This is new work added specifically in this step. After fitting the stoichiometry window, the code checks how much actual capacity (in Ah) that stoichiometry window would deliver, given the Prada2013 electrode geometry:

- Negative electrode (anode): 2.447 Ah
- Positive electrode (cathode): 3.306 Ah

These don't match each other or the target 2.5 Ah - the cathode in particular is oversized relative to the anode. So the active material volume fraction (eps) of each electrode is individually rescaled so that both electrodes deliver exactly 2.5 Ah over their respective stoichiometry windows. This is physically the correct way to fix a capacity mismatch - active material fraction directly controls how much lithium-storing material is packed into the electrode, so scaling it changes capacity without touching unrelated properties like porosity or particle size. A safety check is also included: if scaling would push active material fraction + porosity above 1.0 (which is physically impossible - you can't have more than 100% of the electrode volume), the code prints a warning. No warning was triggered, so this step is physically valid.

Negative particle diffusivity: Increased from $3\text{e-}15$ to $2.5\text{e-}14$ m^2/s . This is a manual tuning step, explicitly allowed by the assignment ("if required, do manual tuning on the current parameter"). Why: the original, smaller diffusivity was causing graphite surface depletion during the high-current (15C) pulses in the HPPC test, which was making the simulation stop early. Raising the diffusivity lets lithium move fast enough within the graphite particles to keep up with the pulses.

Initial SOC: Instead of assuming a fixed number like Step 1&2 did (96.49%), this version backtracks the actual initial SOC from the real first measured HPPC voltage (3.4471 V), using the newly-fitted OCV curve. This gave 98.20% - closer to a real fully-charged HPPC start, and derived from data rather than assumed.

Solver: Switched from CasadiSolver to IDAKLUSolver (falling back to CasADi only if IDAKLU isn't available). IDAKLU is generally a faster, more robust solver for these stiff DFN models over long time horizons, which matters here for the 25-hour HPPC simulation.

New conservation and coverage checks: The code now checks whether lithium is actually conserved between what was electrically pushed into/out of the cell (current $\times$ time) and what changed inside the model's negative electrode. It also explicitly warns if the simulation stops before the full data is covered.

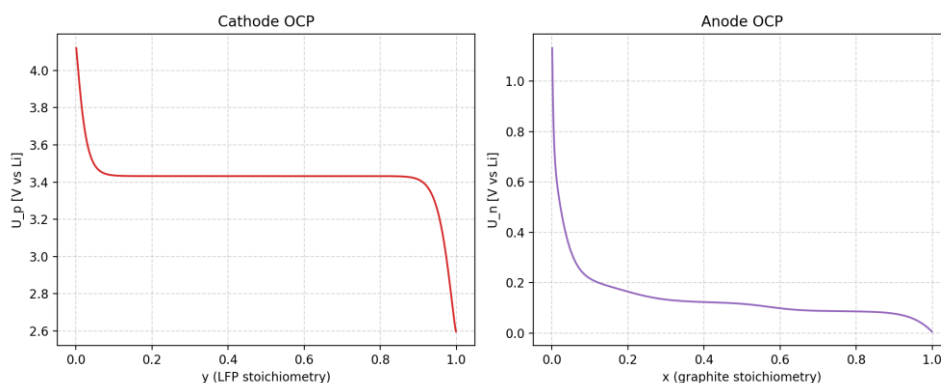
Results

Metric	Step 1&2 (baseline)	Step 3 (this file)
RMSE	113.97 mV	60.14 mV
MAE	65.43 mV	43.17 mV
Max Error	781.83 mV	276.94 mV
R ²	0.262	0.795
Rest-only RMSE	—	52.25 mV
Rest-only bias	—	+34.50 mV
Coverage	100%	100%
Solver	CasADi	IDAKLU

RMSE nearly halved, and R² jumped from 0.26 (barely better than a flat average) to 0.79 (a good fit). This confirms the OCP-curve and capacity fixes were the right diagnosis for the Step 1&2 problem.

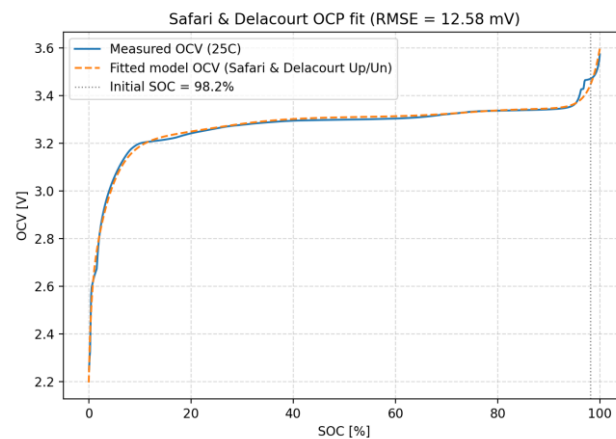
Explaining the plots

OCP sanity check plot (Cathode/Anode OCP):



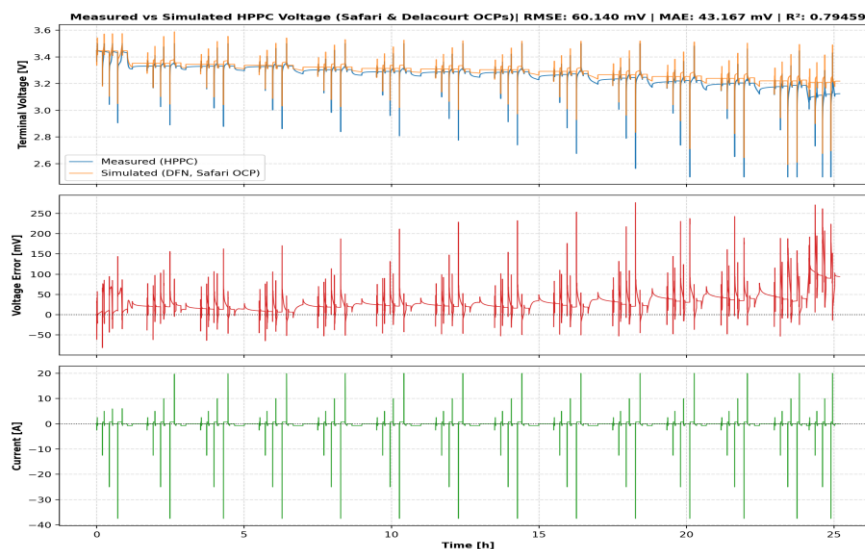
These two curves are just the raw Safari & Delacourt OCP equations plotted across the full stoichiometry range, as a shape check before using them in the model. The cathode (LFP) curve shows the classic LFP "flat plateau" shape around 3.4 V - a long flat middle section with steep rises at both ends. This flat-plateau shape is the well-known signature of LFP chemistry, so seeing it here confirms the equations were coded correctly. The anode (graphite) curve similarly shows graphite's typical shape - steep near $x=0$, then a long shallow decline with small "staircase" steps, which matches published graphite OCP behaviour.

OCV fit check plot:



This overlays the measured OCV table (blue) against what the fitted model produces (orange dashed) across 0–100% SOC. They track each other closely across the full range, including in the flat plateau region (roughly 10–90% SOC) and the steep knee near 100% SOC - which is exactly where the fit matters most, since a lot of SOC change happens with very little voltage change in that plateau. The dotted vertical line marks the backtracked initial SOC (98.2%) used for this HPPC run.

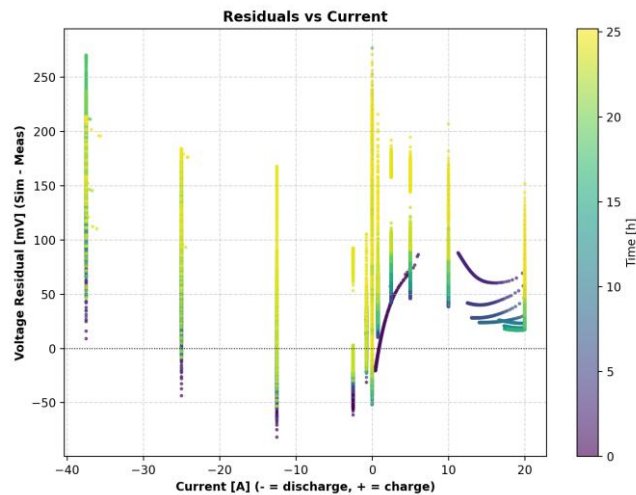
Voltage comparison + error plot:



The simulated line (orange) now tracks the measured line (blue) much more closely for most of the test, especially in the first ~15 hours. The error plot (middle) stays mostly within ± 50 mV early on. It's still not perfect - there's a slow upward drift in the baseline error starting around hour 15–17, and the error still spikes at each current pulse, but the overall match is clearly much better than before.

Residuals vs current plot: Compared to the Step 1&2 version of this same plot, the errors are noticeably smaller and tighter for most current values, especially at low and mid currents. The largest current pulses (near ± 38 A and ± 25 A) still show a wide spread of error

(roughly -50 to +270 mV), especially at later times (yellow-green points), meaning the model still under- or over-shoots during the biggest, latest pulses.



Key observations

1. Replacing the Prada2013 default OCP with literature Safari & Delacourt OCP curves, fitted specifically to our cell's measured OCV, was the single biggest driver of improvement - RMSE dropped by almost half.
2. Fixing the capacity mismatch between the two electrodes (anode was under-capacity, cathode was over-capacity relative to the target 2.5 Ah) removes a hidden source of error that wouldn't show up by just looking at OCV shape alone.
3. The rest-period bias (+34.5 mV, meaning the simulated voltage still sits above the measured voltage during rests) shows there's still a small, systematic OCV mismatch left - the model consistently reads slightly "fuller" than the real cell, especially as the test goes on.
4. The remaining error is concentrated at the highest current pulses and grows later in the test - this points toward the kinetic and diffusion parameters (not yet tuned in this step) as the next thing to fix, since they control how much extra voltage drop happens under high current load.

Discussion

This step confirms the case study's suggested order of attack - fix the OCV/capacity ("manual tuning") before touching kinetics - was the right call. Going from R^2 0.26 to 0.79 with only OCP-curve replacement, stoichiometry fitting, capacity rebalancing, and one diffusivity bump (no full optimization yet) shows how much of the original error was really a "the model doesn't know the right voltage curve or capacity" problem rather than a "the model's electrochemical rate parameters are wrong" problem. The persistent rest-period bias and the growing error at high current and late in the test are the two remaining issues, and they set up exactly what Step 4 (formal optimization of kinetics/diffusion/resistance parameters) needs to address.

Step 3, Part B — step3_charge_discharge.py: Running the Same Fixed Model on the Charge/Discharge Sheets

What this code does

This file takes the exact same model built in step3_safari_ocp.py (same OCP functions, same fitted stoichiometry window, same capacity-matched electrodes, same diffusivity) and runs it separately on the other five data sheets: Charge_1C, Charge_2C, Charge_3C, Charge_4C, and Discharge_1C. This is important because it checks whether the OCV/capacity fix that worked well on HPPC also holds up on plain constant-current charge and discharge tests, which are a different kind of test (long, steady current instead of short alternating pulses).

Methodology and assumptions

Reused model, per-sheet initial SOC: Instead of fitting a new stoichiometry window each time, the code copies the already-fitted parameter_values from step3_safari_ocp.py's setup for every sheet, and only changes the initial SOC for each sheet individually, backtracked from that sheet's own first measured voltage. This is the correct approach - the OCP curves and capacity are cell properties that shouldn't change between tests, but each test naturally starts at a different SOC (a discharge test starts nearly full, a charge test starts nearly empty), so only that piece should change per sheet.

New early-termination diagnostic: This is new compared to step3_safari_ocp.py. For each sheet, the code now explicitly compares:

- how much charge (Ah) the real test moves (calculated from the measured current), against
- how much charge the model can actually deliver from its starting SOC to reach empty or full ($\text{initial_soc} \times 2.5 \text{ Ah}$ for discharge, $(1 - \text{initial_soc}) \times 2.5 \text{ Ah}$ for charge)

If the simulation stops early, the code checks whether the missing time was still under load (meaning the model ran out of capacity before the real cell did) or just rest (meaning only the after-test relaxation period is missing, which is less serious). This diagnostic is valuable because it tells us *why* a simulation stopped early, not just *that* it stopped early.

Same solver, same per-sheet plots and Excel outputs: Each sheet gets IDAKLU solved independently, with its own sheet in the combined Excel output and its own comparison plot, following the same 3-panel layout (voltage / error / current) as DFN model simulation and step3_safari_ocp.py.

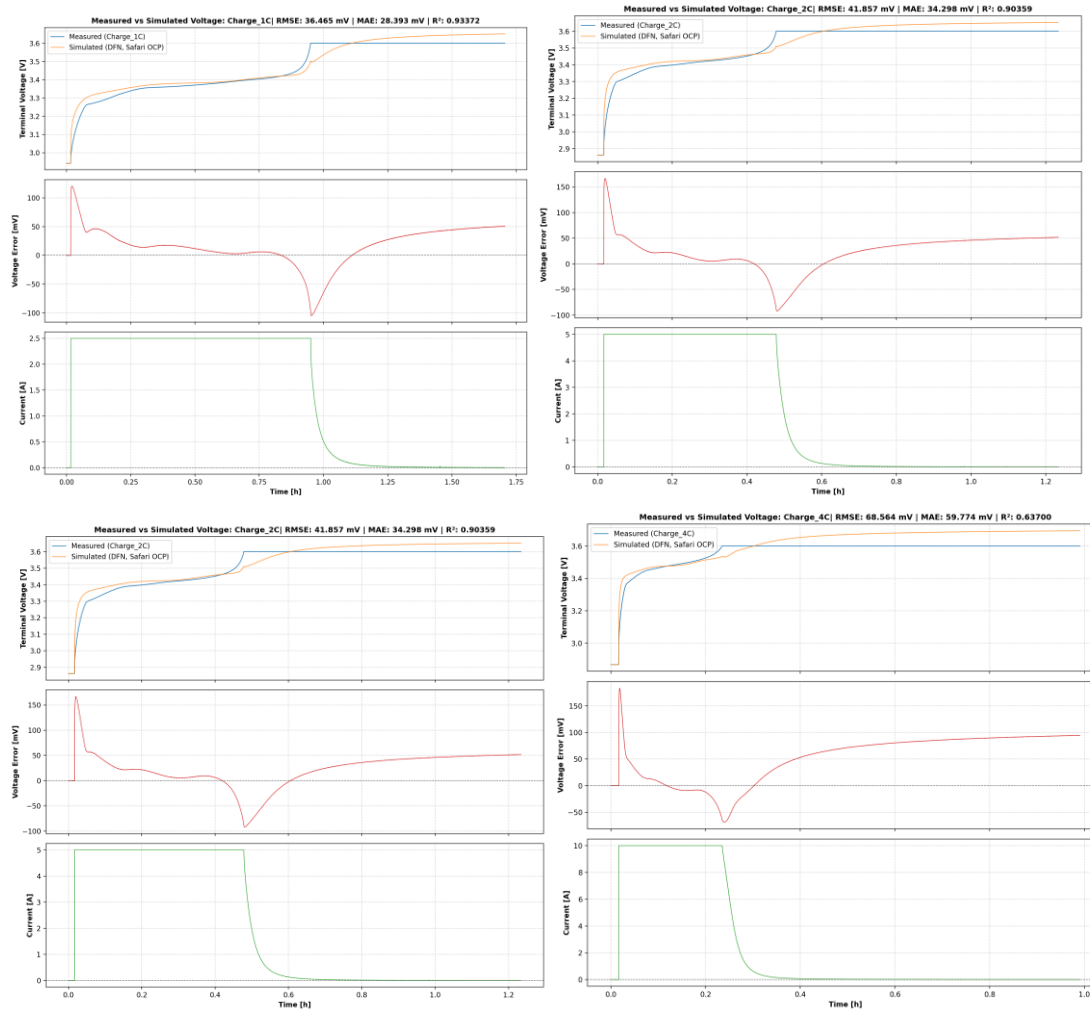
Results

Sheet	RMSE	MAE	Max Error	R ²	Coverage	Rest RMSE	Rest Bias	Termination
Charge_1C	36.46 mV	28.39 mV	120.15 mV	0.934	100%	23.24 mV	+11.51 mV	final time
Charge_2C	41.86 mV	34.30 mV	166.72 mV	0.904	100%	26.20 mV	+13.84 mV	final time
Charge_3C	45.08 mV	37.65 mV	187.72 mV	0.873	100%	31.37 mV	+17.61 mV	final time
Charge_4C	68.56 mV	59.77 mV	183.08 mV	0.637	100%	60.91 mV	+40.58 mV	final time
Discharge_1C	56.67 mV	22.51 mV	643.22 mV	0.711	90.4%	0.97 mV	+0.81 mV	Minimum Voltage (early stop)

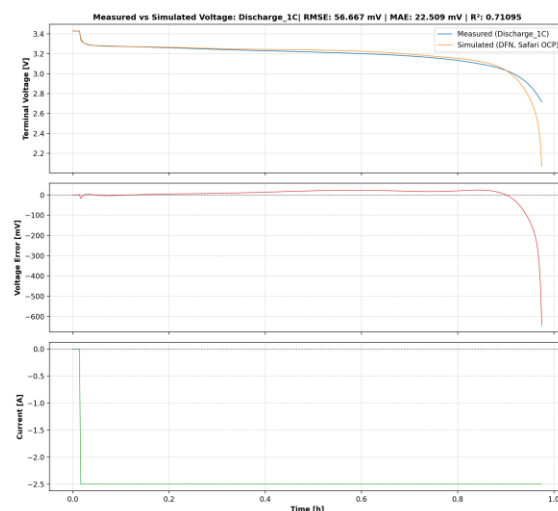
A clear pattern shows up: as the C-rate goes up (1C → 4C), RMSE and rest bias both get consistently worse. Discharge_1C is the one sheet where the simulation didn't fully complete - it stopped early on a "Minimum Voltage" event, at 90.4% coverage.

Explaining the plots

Charge_1C, Charge_2C, Charge_3C, Charge_4C plots: All four follow the same shape. At the very start ($t \approx 0$), there's a sharp voltage jump as current switches on, and the simulated voltage (orange) rises faster than measured (blue) - this shows up as a spike in the error plot right at the start (up to +180 mV for Charge_4C). The error then settles down and stays small for most of the middle of the charge. Near the end, once the real cell hits its ~3.6V charge cutoff, the two lines diverge again - the measured voltage flattens out at 3.6V (charger holding voltage), while the simulated voltage keeps climbing past 3.6V, because the simulation isn't told to cap it there. This causes the growing error visible at the right-hand side of each error plot. This effect gets bigger and starts earlier as C-rate increases (compare Charge_1C's plateau starting near 0.95h vs Charge_4C's near 0.23h), because higher current pushes the cell to its voltage limit faster.



Discharge_1C plot: This one behaves differently and much better for most of the test - the error stays tiny (within about ± 30 mV) for almost the entire 1-hour discharge. Right at the very end, near the discharge cutoff, there's a huge spike (down to -643 mV) where the measured voltage falls off a cliff while the simulated voltage lags slightly behind. This is exactly where the simulation event-stopped ("Minimum Voltage"), so this tail spike is really the last few interpolated points right before the model gave up.



Key observations

1. The same OCV/capacity-fixed model that worked well on HPPC also works reasonably well on plain charge/discharge curves - RMSE stays in the 36–69 mV range for the charge sheets, similar to the HPPC result (60 mV).
2. There's a clear trend: higher C-rate charges have worse RMSE, worse rest bias, and a bigger end-of-charge voltage mismatch. This strongly suggests the kinetic parameters (reaction rates, which matter more at high current) still need tuning - a classic sign the "manual OCV/capacity tuning" step alone isn't enough at fast charge rates.
3. The model doesn't know about the real charger's constant-voltage hold at 3.6 V, so during the tail-end CV phase of each charge, the simulated voltage overshoots past the real cutoff. This isn't really a parameter error - it's a modelling choice (the simulation is driven by measured current, not by a voltage-controlled charger), and it's worth stating clearly in the report as a known limitation rather than something to "fix" by tuning parameters.
4. Discharge_1C's early stop shows the model runs out of usable capacity (or hits a kinetic limit) slightly before the real cell does - this matches the new diagnostic's purpose exactly, and is useful evidence to include in the report as a concrete, explained limitation rather than an unexplained gap.

Discussion

This file is a good validation step - it doesn't add anything new to the model itself, but it stress-tests the Step 3 fix across five very different current profiles (four charge rates plus one discharge), and the results broadly hold up, with RMSE and R^2 values in the same range as the HPPC test. The consistent worsening at higher C-rate is the clearest, most actionable finding here: it tells us exactly what to prioritize in the optimization step - kinetics and possibly contact resistance, which matter most under high current, rather than more OCV or capacity tuning.

Step 4 — DFN parameter optimization.py: Finding the Best Parameters (Optimization)

What this code does

This file is the optimizer. It does not make any plots and it does not run one final "here's the answer" simulation on its own - its whole job is to search through a set of parameters, run the DFN model over and over with different values, compare each run against the real Charge_1C, Charge_2C, Charge_3C, Charge_4C and Discharge_1C data, and slowly walk toward the set of values that gives the smallest error. At the end, it saves the best parameter set it found into `optimized_parameters.json`. The actual voltage-vs-time plots and the "did it work" check come later, in `step5_validate.py`.

It starts from exactly the same base model as Step 3 - same Safari & Delacourt OCP curves, same fitted stoichiometry window, same capacity-matched electrodes, same 2.5 Ah target. Nothing from Step 3 is thrown away; this step only adds a search on top of it.

Methodology and assumptions

Which parameters are being tuned, and why these: The code builds a list of candidate parameters to optimize:

- **Kinetics:** negative and positive electrode exchange-current density (k_n , k_p) - these control how easily the reaction happens at each electrode surface.
- **Diffusion:** negative and positive particle diffusivity (D_n , D_p), plus electrolyte diffusivity (D_e) - these control how fast lithium moves inside the particles and through the liquid electrolyte.
- **Conductivity:** negative and positive electrode conductivity (σ_n , σ_p), plus electrolyte conductivity (κ_e) - how easily electrons/ions move through the solid electrode and the electrolyte.
- **Contact resistance:** a resistance parameter (R_{contact}) that wasn't in Step 3's model at all - it's switched on here for the first time using the "contact resistance": "true" model option.
- **A small OCV correction:** five extra numbers (ocv_c0 , ocv_c1 , ocv_c2 , ocv_end_lo , ocv_end_hi) that let the code make a small adjustment to the shape of the cathode OCP curve - not replace it, just nudge it slightly, mostly near the two ends of the charge/discharge range.

Why 0.03, specifically: The two end-bump terms (ocv_end_lo , ocv_end_hi) use a fixed width of 0.03 in SOC. This number wasn't fitted from the data - it was picked by hand so the bump only has a real effect in roughly the last 10% of SOC at each end (fully charged and fully discharged), which is exactly where the OCP curve is steepest and hardest to fit with a single smooth equation. A narrower width would do almost nothing; a much wider one would start reshaping the flat middle of the curve, which is already a good match.

Each of these is treated as a multiplier on the Step 3 default value (for example, "2× the default diffusivity") rather than an absolute number, and it's fitted in log scale so the search can move it up or down by a factor of 10 in either direction. This is a sensible way to search: battery parameters like diffusivity can span several orders of magnitude, so searching in "times 2, times 5, times 10" steps makes much more sense than searching in plain "+0.001, +0.002" steps.

Why add contact resistance and an OCV correction, when the assignment only mentions kinetics/diffusion/capacity/OCP? The assignment explicitly says OCP-related parameters can be manually tuned, and contact resistance is a standard physical parameter in real cells (a genuine ohmic loss at the connections), so both fit naturally into "grouping parameters by type" as the assignment asks for. The OCV correction is kept deliberately small and shape-constrained - it can only nudge the curve up or down a little near the ends, not redesign it -

and the code has a built-in safety check (explained below) that refuses to let this correction make the OCV curve physically wrong.

How the search actually works - three stages

1. **Sensitivity screening (non-gradient):** before doing any real optimizing, the code nudges each parameter slightly, one at a time, and measures how much that nudge changes the error. Parameters that barely move the error (less than 3% of the largest effect) get frozen - locked at their Step 3 default and left out of the real search. This step exists because with real, noisy, limited data, not every parameter is actually identifiable - you cannot properly fit a parameter that the data barely responds to, and forcing the optimizer to try anyway usually just gives a meaningless answer. Freezing them is the honest choice.
2. **Global sample (non-gradient, Latin Hypercube):** the code picks several random-but-spread-out combinations of the un-frozen parameters (a "Latin Hypercube" is just a way of spreading random samples out evenly instead of clumping randomly) and evaluates all of them. This gives the optimizer a rough map of where the good regions are, so it doesn't get stuck in a poor starting spot.
3. **Local refinement (gradient-based):** starting from the best point found in step 2, the code runs a proper least-squares optimizer (`scipy.optimize.least_squares`, trust-region method) that follows the slope of the error downhill to find a nearby best answer. To keep this fast, instead of recalculating the exact slope (Jacobian) from scratch every single step - which means extra simulations - it updates an approximate slope using a cheaper "Broyden update" trick most of the time, and only recalculates the real slope every 8 steps.

This is a genuine mix of non-gradient methods (sensitivity screening, random sampling) and a gradient-based method (least-squares local search) - exactly what the assignment asks to "assess."

The OCV correction has a safety rule: While searching, the code also checks that its OCV correction never makes the voltage-vs-SOC curve go backwards (i.e., voltage must always increase as SOC increases — a real cell's open-circuit voltage can never decrease as it gets more charged). If a candidate correction would break this rule, it's penalized heavily so the optimizer avoids it. The saved result confirms `"ocv_monotonic": true`, meaning the final answer respects this physical constraint.

Why the optimizer runs in short chunks with saved progress instead of one long run: DFN simulations are expensive, and a normal single-process optimizer that runs for hours risks crashing (or freezing the computer) and losing everything. This code splits the work into short "worker" runs - each one does a limited amount of work (a time budget, an evaluation-count budget, and a memory-use budget), saves every single simulation result to disk as it goes, and then exits cleanly. The next run picks up exactly where the last one left off by reading the saved results back in, so nothing is lost and the computer never has to hold a huge amount of memory at once.

Every result is cached: Before running a new simulation, the code checks whether this exact combination of parameters has already been tried (using a hash of the parameter values), and reuses the saved result if so. This avoids wasting time recomputing something already known.

Results

This file doesn't produce plots, but it does produce a clear before/after number and a final parameter table, saved in `optimized_parameters.json`.

Overall fit improvement:

	Value
Starting cost (Step 3 defaults)	0.00661
Best cost found	0.00171
Number of simulations run	49

(The "cost" here is half the sum of squared voltage errors across all five training sheets — a smaller number means a better fit. Going from 0.00661 to 0.00171 is roughly a 4× reduction.)

What the optimizer actually changed:

Parameter	Change from Step 3	Notes
k_n (anode reaction rate)	×1.05	small increase
D_n (anode diffusivity)	×6.81 (→ $1.70 \times 10^{-13} \text{ m}^2/\text{s}$)	large increase — the anode was letting lithium move much too slowly
D_p (cathode diffusivity)	×0.52 (→ $3.08 \times 10^{-18} \text{ m}^2/\text{s}$)	cut roughly in half
D_e (electrolyte diffusivity)	×2.33 (→ $4.66 \times 10^{-10} \text{ m}^2/\text{s}$)	more than doubled
R_contact	2.19 mΩ	new — this loss wasn't in the model before
OCV correction	c0=+43, c1=-150, c2=+65, end_lo=+149, end_hi=-80 mV	small reshaping of the cathode OCP curve, mostly near the ends

Parameters the data couldn't pin down (frozen at Step 3 defaults): k_p, sig_n, sig_p, kappa_e. Their sensitivity scores (0.0009, 0.0000009, 0.0019, 0.0016) are all tiny next to the

strongest one (D_n , at 0.028), so the optimizer correctly recognized these can't be reliably identified from this particular dataset and left them untouched, rather than guessing.

Values to treat carefully (hit the edge of their allowed search range): ocv_{c1} and ocv_{end_lo} . When an optimizer's answer lands right on the boundary of the range it was allowed to search in, that's usually a sign the "true" best value might actually be even further out - so these two numbers shouldn't be read as fully confirmed, just as "the optimizer wanted to push further than we let it."

Key observations

1. The two diffusivities (D_n and D_e) moved the most, and by a lot - this matches what Step 3's charge-sheet results already hinted at: the model's kinetics/diffusion at high C-rates (2C, 3C, 4C) were clearly too slow compared to the real cell.
2. Adding contact resistance (not present at all in Step 3) picked up a real, physically expected loss - every real cell has some resistance at its terminals and connections, and 2.19 m Ω is a small, believable number for this.
3. Four parameters (k_p , sig_n , sig_p , $kappa_e$) were correctly identified as not fittable from this data and were left alone rather than force-fit - this is a more honest and defensible choice than blindly optimizing every parameter regardless of whether the data actually supports it.
4. The small OCV correction terms landing partly at their search boundary is a signal that the fitted stoichiometry window from Step 3, while already a big improvement, still isn't a perfect match to the real cell's OCV shape right at the very top and bottom of the charge range - worth flagging honestly rather than hiding.
5. The optimizer needed only 49 simulations to get a 4 $\times$ improvement in cost - this is a sign that the mixed non-gradient/gradient-based search strategy converged efficiently, without needing hundreds of brute-force tries.

Discussion

This step covers what the assignment asks for in Step 4: weigh up gradient vs non-gradient methods, build a pipeline, run it, and save the result. The pipeline uses both non-gradient sampling first to find a decent starting point, then a gradient-based local search to fine-tune from there, since gradient methods are fast but can get stuck if they start in a bad spot.

The sensitivity check freezing k_p , sig_n , sig_p , and $kappa_e$ is the most important call here - it stops the optimizer from making up confident numbers for parameters the data can't actually pin down. And the fact that D_n , D_e , and contact resistance moved the most lines up with what the Step 3 residuals already pointed to (slow kinetics at high current, an unmodeled resistance), which is a good sign the fit is real and not just numerical overfitting.

Step 5 - DFN model validation.py: Validating the Optimized Parameters

What this code does

This is the last file in the pipeline. It loads the parameter set saved by Step 4 (optimized_parameters.json), builds the same DFN model with all of those optimized values plugged in, and then runs it on every dataset: the five training sheets (Charge_1C, 2C, 3C, 4C, Discharge_1C). It scores each one, saves an Excel file and plots for each, and also plots some of the model's internal (physical) variables, like anode potential vs lithium and stoichiometry.

Methodology and assumptions

Loading the optimized parameters, not hard-coding them: Instead of retyping the optimized numbers by hand, the code reads them straight out of optimized_parameters.json, the file Step 4 saved. This matters because it removes any chance of a copy-paste mistake between the optimization step and the validation step - whatever Step 4 actually found is exactly what gets tested here.

Applying the parameters with parameter_values.update(): Most parameters are plain numbers, so this is a direct swap. A few parameters in PyBaMM (both exchange-current densities, the electrolyte conductivity, and the positive electrode OCP) aren't stored as plain numbers - they're stored as small functions - so for those specific cases, .update() is given a wrapper function instead of a number. This is explained clearly in the code as the one necessary exception, not a shortcut around the instruction.

Contact resistance turned on: The model option "contact resistance": "true" is switched on here (it wasn't in Step 3), since Step 4 found a real, non-zero value for it.

The OCV correction is "baked in": In Step 4, the small OCV-curve correction was represented as five adjustable numbers that the optimizer could still move. Here in Step 5, those five numbers are fixed at their final optimized values and folded directly into the cathode OCP function. The code re-checks that this final corrected curve is still monotonic (voltage always rising with SOC, never dipping) before using it - the same physical sanity check from Step 4, run one more time on the final numbers.

Initial SOC recalculated with the corrected OCV curve: Since the OCV curve itself changed slightly (because of the correction), the starting SOC for every sheet is backtracked again using this new corrected curve, not the Step 3 one. This keeps everything consistent - the starting point and the OCV curve being used to interpret voltage always match.

Same solver, same scoring, same rules as Step 3: IDAKLU solver, same decimated current input, same RMSE/MAE/Max Error/R² metrics, same rest-only RMSE and

rest bias, same coverage tracking, same lithium-conservation check. Nothing about how the model is judged was changed between steps - only the parameters inside the model changed. This is important because it means the "before vs after" comparison is fair.

Why RMSE is used again here: The same reasoning from Step 2 applies here: RMSE squares the error before averaging, so it penalises a large single miss (like a pulse the model gets badly wrong) more heavily than lots of small ones, which is the right behaviour to reward in a physical model like this. MAE and R^2 are reported alongside it so the numbers can be read together -MAE gives the typical error size, and R^2 gives a sense of how much of the real voltage's up-and-down behaviour the model actually explains.

Extra physical outputs, as required by the assignment: For the two most demanding tests (Charge_4C and the HPPC validation), plus Discharge_1C, the code also pulls out and plots:

- the anode potential vs Li/Li^+ (both at the separator interface and averaged across the electrode) - with a red dashed line marking 0 V, which is the lithium-plating danger threshold,
- the cathode potential vs Li/Li^+ ,
- the surface stoichiometry of both the graphite anode and the LFP cathode.

Why HPPC is treated differently: Every other sheet is a training sheet - Step 4 used it to fit the parameters. HPPC was deliberately kept out of that fitting, so running it here is a genuine test of whether the optimized parameters generalize to data the model has never seen, not just data it was tuned to match. The code prints this distinction explicitly in its output.

Results

Training sheets (used to fit the parameters) vs the Step 4 training result - confirms the parameters still work when actually applied end-to-end:

Sheet	RMSE	MAE	R^2	Rest RMSE	Rest bias	Coverage
Charge_1C	26.77 mV	18.50 mV	0.964	5.82 mV	-2.87 mV	100%
Charge_2C	35.32 mV	29.16 mV	0.931	15.88 mV	-8.61 mV	100%
Charge_3C	35.00 mV	29.91 mV	0.923	17.43 mV	-9.87 mV	100%
Charge_4C	15.81 mV	10.35 mV	0.981	1.65 mV	+0.74 mV	100%
Discharge_1C	28.40 mV	11.84 mV	0.947	0.97 mV	+0.81 mV	91.7%

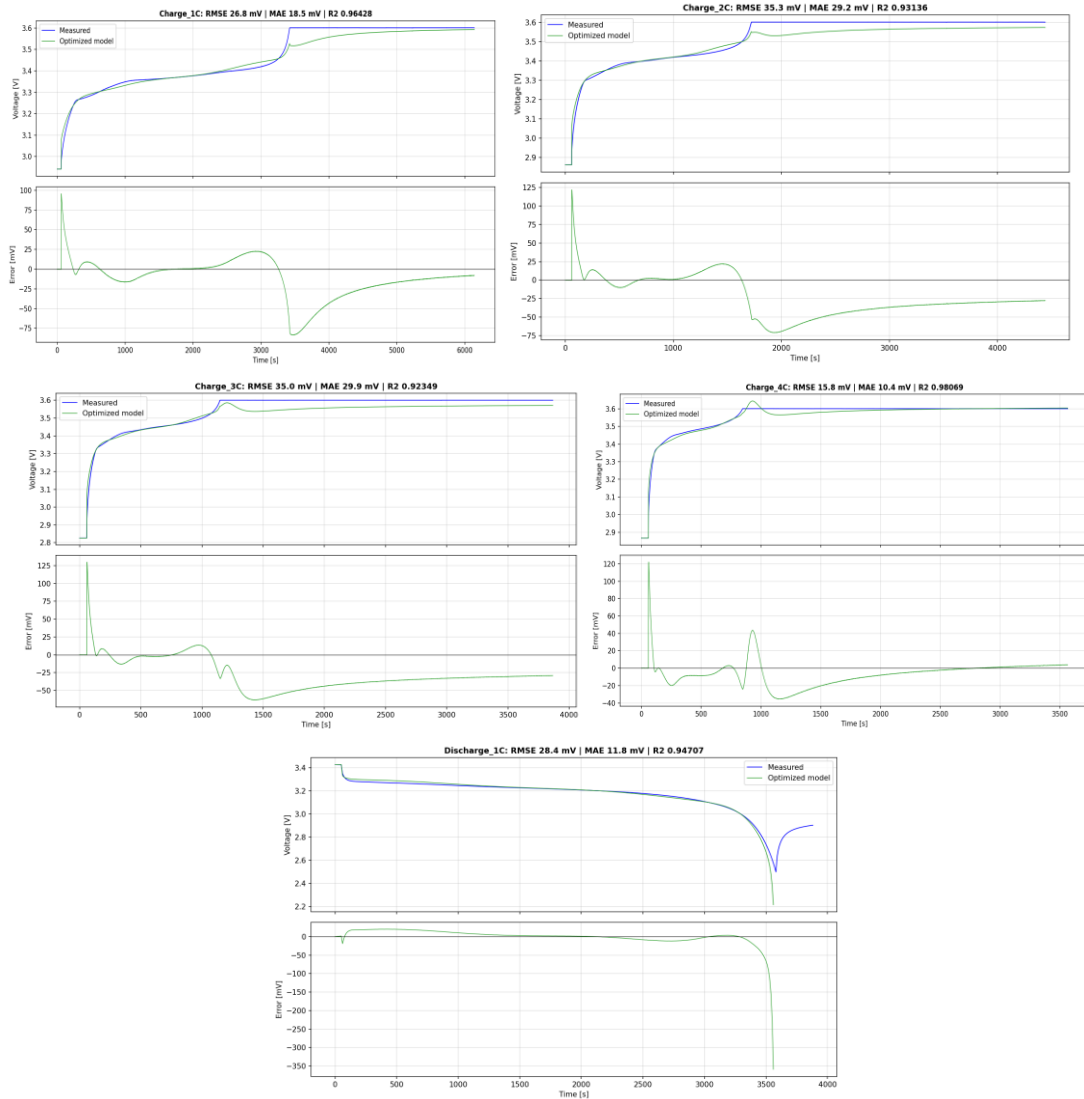
HPPC — the real validation test, never used in fitting:

Metric	Value
RMSE	55.46 mV

Metric	Value
MAE	38.95 mV
Max Error	417.65 mV
R^2	0.825
Rest RMSE	50.85 mV
Rest bias	+25.09 mV
Pulses ≤ 10 A RMSE	58.37 mV
Pulses > 10 A RMSE	79.93 mV
Coverage	100%
Li conservation gap	0.022 mAh

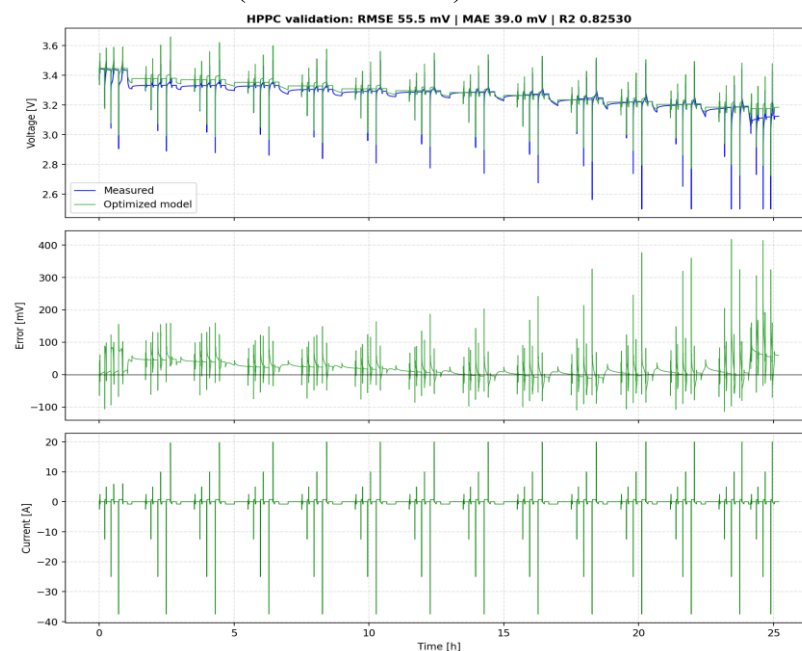
The R^2 of 0.825 on HPPC is a strong number for data the optimizer never saw - it went from R^2 0.795 with the plain Step 3 model to 0.825 here, and RMSE improved too. This confirms the optimized parameters aren't just overfitted to the training sheets; they generalize reasonably well.

Explaining the plots



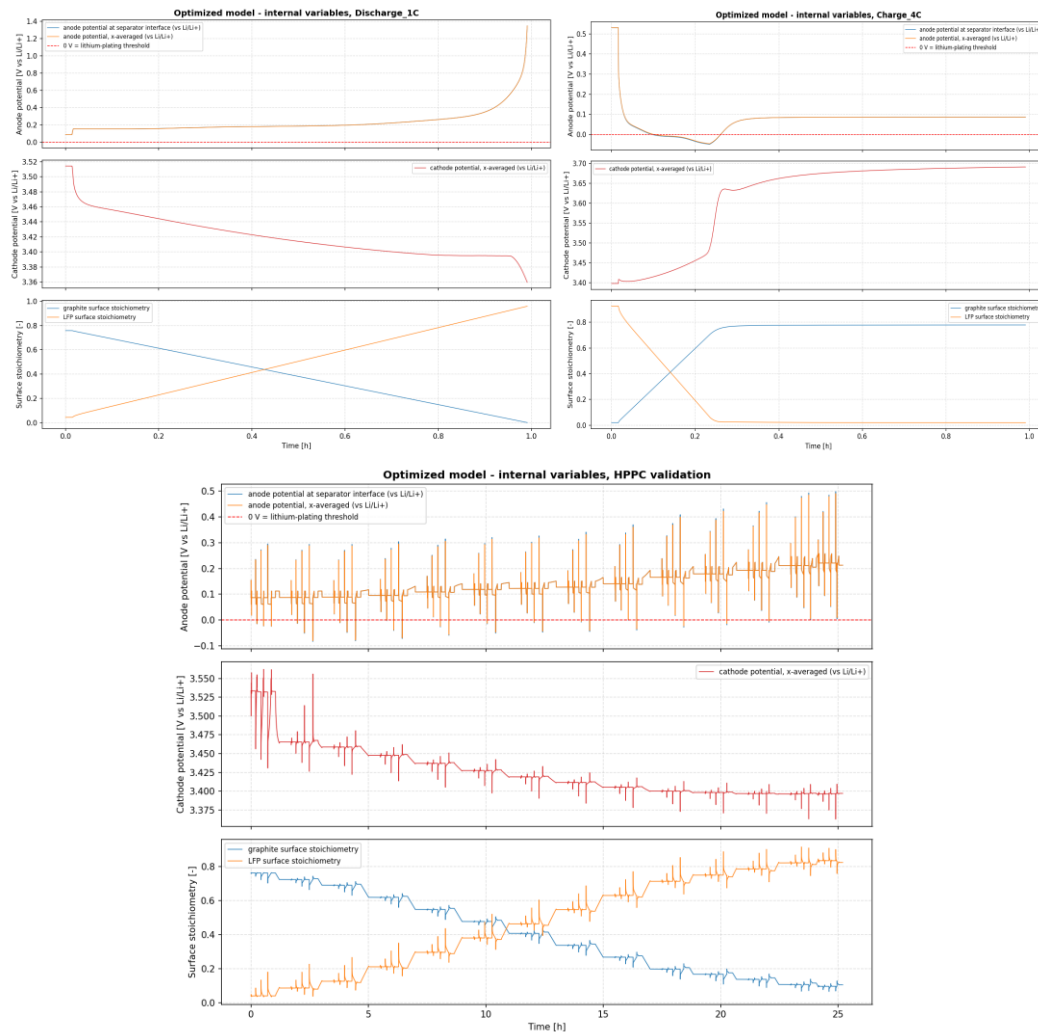
The five training-sheet plots (Charge_1C–4C, Discharge_1C): All four charge plots show the same shape: a sharp error spike right at the very start (up to about +95 to +130 mV) as the current switches on, then the error settles to within roughly ± 20 mV for most of the middle of the charge, then a second bump appears near the top-of-charge knee (where the real charger holds the cell at 3.6 V while the model keeps climbing slightly past it, as noted back in Step 3). For Charge_1C and Charge_4C the error works its way back down close to zero by the end of the test; for Charge_2C and Charge_3C it narrows but is still sitting around -20 to -30 mV at the last plotted point, so "settles again" is a bit generous for those two - the rest-only bias (-8.6 mV and -9.9 mV) is smaller because it's measured only over the handful of truly zero-current samples right at the very end, not the whole CV-hold tail. This pattern - start-up transient, then good tracking, then the known CV-hold mismatch at the end - repeats across all four C-rates and is expected behavior given the current-driven (not voltage-controlled) simulation setup. Discharge_1C tracks almost perfectly for nearly the whole hour (error mostly within ± 20 mV) and only diverges sharply right at the very end, where the real cell dips to a low-voltage minimum and then recovers slightly, but the model - driven only by measured current - doesn't get told about that final rest-and-recovery segment, so it just keeps discharging voltage down.

HPPC voltage/error plot: The green (optimized) line tracks the blue (measured) line noticeably better, especially in the first 10–15 hours where the error mostly stays within about ± 50 mV outside of the big pulses. There's still a slow upward drift in the baseline error as time goes on, and the pulse spikes (especially the largest current pulses) still reach up to 300–400 mV late in the test - this matches the rest bias of +25 mV and the fact that pulses above 10 A (79.9 mV RMSE) are still noticeably worse than pulses at or below 10 A (58.4 mV RMSE).



Physical variable plots (anode/cathode potential, stoichiometry): For

Discharge_1C, the anode potential stays comfortably above the red 0 V line for the whole test - no lithium-plating risk there. Charge_4C and the HPPC run are a different story: in Charge_4C the anode potential dips below 0 V (down to about -0.05 V) for a short window around 0.1–0.25 hours right at the start of the fast charge, before recovering and settling around $+0.08$ V. In the HPPC plot the anode potential also touches or briefly dips just below 0 V at several of the larger current pulses spread across the test - not just once. The pattern is clear: the anode gets close to or briefly below the plating threshold specifically during the largest current pulses, in both the fast-charge test and the HPPC pulses, not only in one isolated spot. This is a real, physically meaningful flag - it says high-current pulses/charges briefly push the anode into plating-risk territory even with the optimized parameters. The stoichiometry plots behave exactly as expected - during charge, graphite stoichiometry rises while LFP stoichiometry falls (lithium moving from cathode to anode); during discharge, it's the reverse. In the HPPC plot, both stoichiometries drift steadily over the 25 hours as the pulses slowly deplete the cell, which lines up with the SOC drift already seen in the voltage plots.



Key observations

1. The optimized parameters hold up well outside the data they were fitted on - HPPC's R^2 improved from 0.795 (Step 3) to 0.825 here, which is the clearest evidence that the optimization genuinely improved the model.
2. The remaining HPPC error is concentrated in two places: the slow rest-voltage drift (rest bias +25 mV, still not zero) and the large current pulses (>10 A RMSE is 80 mV vs 58 mV for smaller pulses) - both of these were already flagged as open issues at the end of Step 4, and this step confirms they weren't fully resolved by the optimization.
3. The anode potential dipping below 0 V - at the start of the Charge_4C test, and at several of the larger pulses in the HPPC run - is a genuine, useful physical finding: it shows the model is at some risk of lithium plating specifically under high-current conditions, which is exactly the kind of insight this whole physical model is meant to surface.
4. Discharge_1C's simulation doesn't reach 100% coverage (91.7%) because the real test includes a short rest-and-recovery period at the very end that the model, driven only by the measured current, doesn't reproduce - this is a data/setup limitation, not a sign the parameters are wrong.

Discussion

This final step closes the loop the case study asks for: take the parameters found in Step 4, apply them properly with `parameter_values.update()`, and check whether they actually work - both on the data used to find them and, more importantly, on the HPPC data that was deliberately kept separate the whole time. The result is a genuine, honest validation: the model clearly improved (R^2 up from 0.262 at the very start, to 0.795 after the Step 3 OCV/capacity fix, to 0.825 after full optimization), but it isn't perfect, and the rest-voltage drift and the high-current pulse error are both still visible and reported with real numbers. The lithium-plating check during Charge_4C is an extra finding that goes beyond just matching voltage numbers.